

Red Team vs Blue Team Attack & Defense Simulation

Solo Execution — One Student, Both Attacker and Defender

RED TEAM	BLUE TEAM	LEGITIMATE USER
Kali Linux 192.168.152.3 Nmap • Dirb • Curl	Linux Lite 6.6 192.168.152.5 Apache • UFW • AWK	Linux Mint 22.3 192.168.152.6 Firefox • Curl

Student Name	Saidinesh Andekar
Batch	13
Date	17 March 2026
Lab Type	Red Team vs Blue Team — Attack & Defense Simulation
Platform	Oracle VirtualBox, Host-Only Isolated Network
Server	Apache HTTP Server 2.4.52 (Ubuntu) on Linux Lite 6.6
Attack Tools	Nmap 7.98 Dirb v2.22 Curl 8.18.0
Defense Tools	UFW 0.36.1 Apache Access Logs AWK / Sort / Uniq
Attack Volume	19,369 HTTP requests generated from attacker IP
Final Result	Attacker blocked — Legitimate user unaffected

Contents

1	Project Overview	4
1.1	System Architecture	4
1.2	Lab Network Diagram	4
1.3	Tools at a Glance	5
2	Virtual Machine Setup and Installation	6
2.1	VM 1 — Kali Linux (Red Team / Attacker)	6
2.1.1	Command 1: whoami	6
2.1.2	Command 2: id	7
2.1.3	Command 3: ip a	8
2.1.4	Command 4: ping -c 4 192.168.152.5	8
2.1.5	Command 5: curl http://192.168.152.5 (pre-attack check)	9
2.2	VM 2 — Linux Lite 6.6 (Blue Team / Apache Web Server)	11
2.2.1	Command 6: ip a, whoami, id on Linux Lite (Screenshot 6)	11
2.2.2	Command 7: sudo apt-get install apache2 -y	12
2.2.3	Command 8: sudo systemctl status apache2 (Screenshot 7)	12
2.2.4	Command 9: cd /var/www/html, ls, sudo cat index.html (Screenshot 6)	13
2.3	VM 3 — Linux Mint 22.3 Xfce (Legitimate Client)	15
2.3.1	Command 10: whoami, id, ip a on Linux Mint (Screenshot 11)	15
3	Red Team Activities — Attack Simulation	16
3.1	Attack 1 — Port Scanning with Nmap (Screenshot 2, Kali 20:37)	16
3.2	Attack 2 — Directory Enumeration with Dirb (Screenshot 2, Kali 20:37)	17
3.3	Attack 3 — Traffic Flood with Curl (Screenshot 2, Kali 20:37)	18
4	Legitimate User Activity — Linux Mint	19
5	Blue Team Activities — Detection and Defense	20
5.1	Step 1 — Real-Time Log Monitoring (Screenshots 5 and 7)	20
5.2	Step 2 — Attacker Identification with AWK (Screenshots 7 and 8)	21
5.3	Step 3 — Install and Verify UFW (Screenshot 8)	22
5.4	Step 4 — Block the Attacker with UFW (Screenshots 9 and 10)	23
6	Results and Verification	25
6.1	Kali Blocked — curl Test (Screenshot 3)	25
6.2	Kali Blocked — Firefox Browser (Screenshot 4)	26
6.3	Linux Mint Unaffected — Firefox Success (Screenshot 12)	27

6.4	Final Test Results	27
7	Conclusion and Key Learnings	29
7.1	Technical Skills Demonstrated	29
7.2	Key Learnings	29
7.3	Real-World Application	30

1 Project Overview

Real-World Scenario — The TechCorp Incident

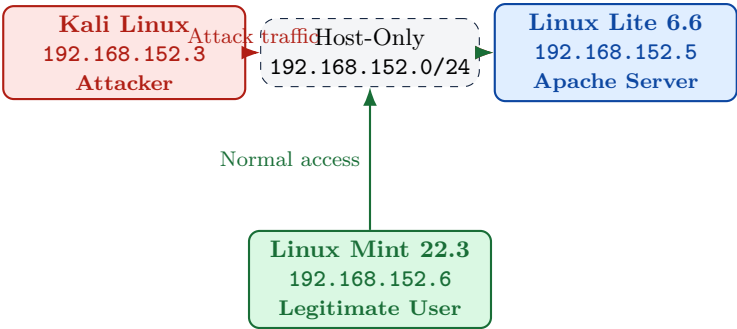
TechCorp runs an internal Apache web server at `192.168.152.5` used by all 500 employees daily. On 17 March 2026 at 20:37, the SOC analyst notices the access logs flooding with thousands of identical automated requests from `192.168.152.3`. The analyst must confirm the attack, identify the source, block it at the firewall, and verify that the HR manager at `192.168.152.6` can still work normally — without taking the server offline.

This lab replicates that exact lifecycle in a controlled VirtualBox environment.

1.1 System Architecture

Role	Machine	IP Address	Purpose
Red Team	Kali Linux	<code>192.168.152.3</code>	Attacker — scan, enumerate, flood
Blue Team	Linux Lite 6.6	<code>192.168.152.5</code>	Apache server + log monitor + fire-wall
Legitimate User	Linux Mint 22.3	<code>192.168.152.6</code>	Normal employee browsing the server

1.2 Lab Network Diagram



1.3 Tools at a Glance

Tool	Version	Team	Real-World Analogy
<code>nmap</code>	7.98	Red	Burglar testing which doors and windows are unlocked
<code>dirb</code>	2.22	Red	Locksmith trying every key combination to find hidden rooms
<code>curl</code>	8.18.0	Red	Robot pressing a doorbell thousands of times per second
Apache2	2.4.52	Blue	Reception desk that handles all visitor requests
<code>tail -f</code>	—	Blue	Security guard watching live CCTV footage as events unfold
<code>awk</code> + <code>sort</code>	—	Blue	Analyst counting how many times each visitor appears on camera
<code>ufw</code>	0.36.1	Blue	Security system that bans one specific face from entering

2 Virtual Machine Setup and Installation

Lab Environment Design

All three VMs use Oracle VirtualBox with a **Host-Only Adapter** (192.168.152.0/24), creating a fully isolated private network. No attack traffic reaches the internet. A second adapter (NAT) was temporarily added to Linux Lite only to download Apache packages, then removed before the attack phase.

2.1 VM 1 — Kali Linux (Red Team / Attacker)

Kali Linux — Pre-Installed Penetration Testing Distribution

No fresh installation required. Kali was already configured in VirtualBox with all attack tools pre-loaded. It is the industry standard for ethical hackers and certified penetration testers (OSCP, CEH, PNPT).

Parameter	Value
Version	Kali Linux Rolling (pre-installed)
Username	cipherwolf
RAM	2525 MB
Network	Adapter 1 = Host-Only Adapter (vboxnet0)
IP Address	192.168.152.3 (eth0)
Tools	Nmap 7.98, Dirb 2.22, Curl 8.18.0, Wireshark

2.1.1 Command 1: whoami

Definition: whoami

Full Form: Who Am I **Category:** Built-in Unix shell command

Syntax: whoami (no arguments required)

Definition: Prints the username of the currently logged-in user by reading the effective user ID (EUID) of the current process and looking it up in `/etc/passwd`. It answers one simple question: *which account is running this terminal session?*

Why run first: A penetration tester must confirm their own identity before any operation. Running as the wrong user (e.g. accidentally as root) invalidates the test.

Real-World Scenario: Airport Security Officer Badge Check

Before a security officer begins their shift at the airport, they scan their own badge into the system. If they accidentally log in on a colleague's badge, every action gets attributed to the wrong person — a serious compliance failure. In exactly the same way, the penetration tester runs `whoami` first to confirm they are operating as `cipherwolf`, the account authorised for this test. If the test report shows the wrong username, the entire engagement is called into question.

```
(cipherwolfkali)-[~]  
$ whoami  
cipherwolf
```

2.1.2 Command 2: id**Definition: id**

Full Form: Identity **Category:** GNU coreutils

Syntax: `id [username]`

Definition: Displays the numeric user ID (UID), primary group ID (GID), and the complete list of all supplementary groups the current user belongs to. Groups determine which system resources and commands the user can access.

Key output fields in this lab:

- `uid=1000(cipherwolf)` — user number 1000, username cipherwolf
- `27(sudo)` — member of the sudo group; can run root commands
- `124(wireshark)` — permitted to capture network packets
- `131(kaboxer)` — can run Kali attack tool containers

Real-World Scenario: Employee Department Access Card at TechCorp

Every TechCorp employee receives an access card (UID) and is assigned to departments (groups): IT, Security, Finance. The card determines which doors open for them. The `id` command reads all department memberships on that card. The presence of `sudo` is like having a master key — without it, Nmap and Dirb would fail because they need elevated privileges to craft raw network packets. In a real penetration test, auditors record the `id` output as proof of the privilege level the test was conducted under.

```
(cipherwolfkali)-[~]  
$ id  
uid=1000(cipherwolf) gid=1000(cipherwolf) groups=1000(cipherwolf),4(adm),  
20(dialout),24(cdrom),25(floppy),27(sudo),29,0(users),101(netdev),  
103(scanner),116(bluetooth),121(lpadmin),124(wireshark),  
130(vboxsf),131(kaboxer)
```

2.1.3 Command 3: ip a

Definition: ip a

Full Form: ip address show (iproute2 utility)

Category: Linux networking command

Syntax: ip a or ip address show

Definition: Lists all network interfaces on the machine together with their assigned IPv4/IPv6 addresses, MAC hardware addresses, MTU size, and connection state (UP = active). Replaced the older ifconfig command on modern Linux.

Key output lines:

- eth0 state UP — the interface is active and connected
- inet 192.168.152.3/24 — the attacker's IP on the lab network
- brd 192.168.152.255 — broadcast address for the subnet

Real-World Scenario: Finding Your Own Warehouse Address Before Shipping Parcels

A courier company must know their own depot address before they can track which deliveries originated from them. The penetration tester records their own IP (**192.168.152.3**) for two critical reasons: (1) to confirm they are on the correct network segment to reach the target, and (2) to prove in the audit report which IP generated all the attack traffic. At TechCorp, the SOC analyst will see **192.168.152.3** appearing thousands of times in the access logs. The tester's ip a output is the legal proof that this IP was theirs during the authorised test window.

```
(cipherwolfkali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:21:ee:d9 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.152.3/24 brd 192.168.152.255 scope global dynamic noprefixroute eth0  
        valid_lft 419sec preferred_lft 419sec  
    inet6 fe80::a00:27ff:fe21:eed9/64 scope link noprefixroute
```

2.1.4 Command 4: ping -c 4 192.168.152.5

Definition: ping -c 4 <target-IP>

Full Form: Packet Internet Groper

Category: Network diagnostic utility (uses ICMP protocol)

Syntax: ping -c <count> <IP>

Definition: Sends ICMP Echo Request packets to a remote host and waits for ICMP Echo Reply responses. Measures round-trip time and packet loss to verify basic network reachability

before running heavier tools.

Flag: `-c 4` — send exactly 4 packets then stop. Without `-c`, ping runs forever.

Output:

- `icmp_seq=1 ttl=64 time=4.48 ms` — packet 1 returned in 4.48 ms
- `0% packet loss` — all 4 packets reached the target and returned

Real-World Scenario: Calling a Client Before Driving Three Hours to Visit Them

A sales representative never drives three hours to a client's office without calling first to confirm they are available. Running `ping` before any attack tool is exactly that call. If 0% packet loss is returned, the target server is online and the network path is confirmed. If ping failed, running Nmap would waste minutes producing empty results. In professional penetration test reports, the ping result is documented as *Host Discovery* evidence — the first line proving the target was alive when the test began.

```
(cipherwolfkali)-[~]
$ ping -c 4 192.168.152.5
PING 192.168.152.5 (192.168.152.5) 56(84) bytes of data.
64 bytes from 192.168.152.5: icmp_seq=1 ttl=64 time=4.48 ms
64 bytes from 192.168.152.5: icmp_seq=2 ttl=64 time=2.79 ms
64 bytes from 192.168.152.5: icmp_seq=3 ttl=64 time=2.22 ms
64 bytes from 192.168.152.5: icmp_seq=4 ttl=64 time=3.02 ms

--- 192.168.152.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3257ms
rtt min/avg/max/mdev = 2.221/3.126/4.476/0.831 ms
```

2.1.5 Command 5: `curl http://192.168.152.5` (pre-attack check)

Definition: `curl <URL>`

Full Form: Client URL

Category: Command-line HTTP/HTTPS client (libcurl library)

Syntax: `curl [options] <URL>`

Definition: Sends an HTTP request to a web server and prints the raw response body (usually HTML source code) to the terminal. Supports GET, POST, PUT, DELETE, and all other HTTP methods. Used to manually fetch pages, test APIs, and probe server responses without opening a browser.

Real-World Scenario: Reading the Notice Board Before Entering TechCorp's Building

Before a visitor enters TechCorp, they read the notice board at reception to understand what the company does and what floor to go to. `curl` is the penetration tester's notice-board read — it fetches exactly what the server shows to any visitor. The HTML response reveals the page title, technologies used, and confirms Apache is serving content before heavier tools like Nmap are launched.

```
(cipherwolfkali)-[~]  
$ curl http://192.168.152.5  
<!DOCTYPE html>  
<html>  
<head>  
  <title>Red vs Blue Team Simulation</title>  
</head>  
<body style="font-family: Arial; text-align: center; background-color: #f4f4f4;">  
  <h1> Cyber Security Lab Project</h1>  
  <h2>Red Team vs Blue Team Simulation</h2>  
  <p><strong>Name:</strong> Saidinesh</p>  
  <p><strong>Batch:</strong> 13</p>  
  <hr>
```

2.2 VM 2 — Linux Lite 6.6 (Blue Team / Apache Web Server)

Linux Lite 6.6 — Ubuntu-Based Web Server and Defender Machine

Freshly downloaded and installed. Acts as both the Apache web server (attack target) and the Blue Team defense console. Ubuntu 22.04 base means all **apt** packages including Apache2 and UFW install cleanly.

Parameter	Value
OS	Linux Lite 6.6 (Ubuntu 22.04 LTS base)
ISO	linux-lite-6.6-64bit.iso from linuxliteos.com (2.4 GB)
RAM / Disk	1024 MB / 25 GB VDI Dynamically Allocated
Network	Adapter 1 = Host-Only + Adapter 2 = NAT (temp, for apt)
IP Address	192.168.152.5 (enp0s3) and 10.0.3.15 (enp0s8 NAT)

VirtualBox installation steps:

1. Downloaded `linux-lite-6.6-64bit.iso` (2.4 GB) from linuxliteos.com
2. New VM: Type = Linux, Version = Ubuntu (64-bit), RAM = 1024 MB, HDD = 25 GB VDI
3. Settings → Storage → Optical Drive → attached ISO
4. Settings → Network → Adapter 1 = Host-Only Adapter (vboxnet0)
5. Settings → Network → Adapter 2 = NAT (temporarily enabled for **apt**)
6. Booted VM → GRUB → **Direct Install Linux Lite 6.6** → Erase disk → username: user, password: 1234
7. After Apache installed — **disabled Adapter 2 (NAT)** to enforce lab isolation

2.2.1 Command 6: `ip a`, `whoami`, `id` on Linux Lite (Screenshot 6)

```

user ~ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:00:bf:21 brd ff:ff:ff:ff:ff:ff
    inet 192.168.152.5/24 brd 192.168.152.255 scope global dynamic noprefixroute enp0s3
        valid_lft 402sec preferred_lft 402sec
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:34:20:49 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 85902sec preferred_lft 85902sec

user ~ whoami
user

user ~ id
uid=1000(user) gid=1000(user) groups=1000(user),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),114(lpadmin),115(sambashare)

```

2.2.2 Command 7: `sudo apt-get install apache2 -y`

Definition: `sudo apt-get install apache2 -y`

sudo — Superuser Do. Runs the command with root (administrator) privileges. Required because installing software writes to system directories a normal user cannot modify.

apt-get install — Advanced Package Tool. Downloads a package and all its dependencies from Ubuntu's official repository and installs them.

apache2 — The package name for Apache HTTP Server v2, the world's most widely deployed open-source web server (powers over 30% of all websites globally).

-y — Automatically confirms all y/n prompts during installation.

Real-World Scenario: Hiring a Receptionist for TechCorp's Office

Before Apache is installed, any request to `192.168.152.5` receives no response — like knocking on an office door with no one inside. Installing Apache2 is like hiring a fully trained receptionist who sits at the front desk 24/7 and hands every visitor the web page from `/var/www/html/`. The `apt-get` system is like a staffing agency that delivers a pre-vetted worker with all required equipment (dependencies) already included.

```
user ~ sudo apt-get update
user ~ sudo apt-get install apache2 -y
Reading package lists... Done
Building dependency tree... Done
...
Setting up apache2 (2.4.52-1ubuntu4.6) ...
Processing triggers for ufw (0.36.1-4ubuntu0.1) ...
```

2.2.3 Command 8: `sudo systemctl status apache2` (Screenshot 7)

Definition: `sudo systemctl status apache2`

systemctl — System Control. The command used to manage Linux services (background processes) in systems running `systemd`.

status — Shows whether the service is running, when it started, the process ID (PID), memory and CPU usage, and the most recent log lines.

apache2 — The service name for Apache HTTP Server.

Related commands used:

- `systemctl start apache2` — starts the service immediately
- `systemctl enable apache2` — auto-starts on every reboot

Key indicator: Active: active (running) in green = server is live.

Real-World Scenario: CCTV System Health Check Before the Security Shift Begins

Before a security guard begins monitoring TechCorp's cameras, they run a health check to confirm all cameras are recording. `systemctl status apache2` is that health check. The green **active (running)** output confirms Apache's recording is live. The **enable** flag ensures the camera automatically restarts after any power cut (server reboot), so there is never a gap in the surveillance record.

```
user /var/www/html sudo systemctl status apache2
[sudo] password for user:
apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset)
   Active: active (running) since Tue 2026-03-17 19:38:01 IST; 35min ago
     Docs: https://httpd.apache.org/docs/2.4/
  Process: 1663 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
 Main PID: 1700 (apache2)
    Tasks: 55 (limit: 992)
   Memory: 8.5M
      CPU: 21.367s
   CGroup: /system.slice/apache2.service
           |-1700 /usr/sbin/apache2 -k start
           |-1701 /usr/sbin/apache2 -k start
           |-1702 /usr/sbin/apache2 -k start

Mar 17 19:37:58 user-VirtualBox systemd[1]: Starting The Apache HTTP Server...
Mar 17 19:38:01 user-VirtualBox apachectl[1689]: AH00558: apache2: Could not re...
Mar 17 19:38:01 user-VirtualBox systemd[1]: Started The Apache HTTP Server.
```

2.2.4 Command 9: `cd /var/www/html, ls, sudo cat index.html` (Screenshot 6)

Definition: `cd /var/www/html | ls | sudo cat index.html`

/var/www/html — Apache's default document root. Any file placed here is automatically served when a browser or `curl` requests the server's IP.

cd — Change Directory. Navigates the terminal to that folder.

ls — List. Shows all files in the current directory.

sudo cat index.html — Prints the full content of the homepage to the terminal. Confirms the lab's custom web page is present and correct.

```
user ~ cd /var/www/html
user /var/www/html ls
index.html

user /var/www/html sudo cat index.html
<!DOCTYPE html>
<html>
<head>
  <title>Red vs Blue Team Simulation</title>
</head>
<body style="font-family: Arial; text-align: center; background-color: #f4f4f4;">
  <h1> Cyber Security Lab Project</h1>
  <h2>Red Team vs Blue Team Simulation</h2>
  <p><strong>Name:</strong> Saidinesh</p>
```

```
<p><strong>Batch:</strong> 13</p>
<h3> Red Team Activities</h3>
<p>Performed Reconnaissance using Nmap</p>
<p>Directory Enumeration using Dirb</p>
<p>Simulated Attack using Curl Requests</p>
<h3> Blue Team Activities</h3>
<p>Monitored Logs using Apache Access Logs</p>
<p>Detected Attacker IP</p>
<p>Blocked Attacker using Firewall (UFW)</p>
<h3> Simulation Note</h3>
<p>This project was performed individually by acting as both attacker and defender
.</p>
</body>
</html>
```

How this page looks in Firefox — Linux Mint browsing the server (Screenshot 12):



2.3 VM 3 — Linux Mint 22.3 Xfce (Legitimate Client)

Linux Mint 22.3 — The Normal Employee

Freshly downloaded and installed. Represents TechCorp's HR manager checking the company portal. Its 5 requests vs the attacker's 19,369 makes detection immediate. After the UFW block, this VM proves the firewall rule is surgically precise.

Parameter	Value
OS	Linux Mint 22.3 Xfce Edition
ISO	linuxmint-22.3-xfce-64bit.iso from linuxmint.com (2.8 GB)
RAM / Disk	1024 MB / 25 GB VDI
Network	Adapter 1 = Host-Only Adapter (same vboxnet0)
IP Address	192.168.152.6 (enp0s3) and 10.0.3.15 (enp0s8)

Installation steps (same procedure as Linux Lite, without NAT adapter):

1. Downloaded `linuxmint-22.3-xfce-64bit.iso` (2.8 GB) from linuxmint.com
2. New VM: Type = Linux, Version = Ubuntu (64-bit), RAM = 1024 MB, HDD = 25 GB VDI
3. Settings → Network → Adapter 1 = Host-Only Adapter only (no NAT needed)
4. Booted → **Start Linux Mint** from GRUB → live desktop loaded
5. Ran `ubiquity` in terminal → Erase disk → username: user, password: 1234

2.3.1 Command 10: whoami, id, ip a on Linux Mint (Screenshot 11)

```

user@user-VirtualBox:~$ whoami
user

user@user-VirtualBox:~$ id
uid=1000(user) gid=1000(user) groups=1000(user),4(adm),24(cdrom),
27(sudo),30(dip),46(plugdev),100(users),105(lpadmin),125(sambashare)

user@user-VirtualBox:~$ ip a
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 08:00:27:9f:c1:c0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.152.6/24 brd 192.168.152.255 scope global dynamic noprefixroute enp0s3
        valid_lft 451sec preferred_lft 451sec
    inet6 fe80::501e:284:7d64:bcb4/64 scope link noprefixroute
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 08:00:27:ca:f0:46 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 86252sec preferred_lft 86252sec

```

3 Red Team Activities — Attack Simulation

Red Team Role

The Red Team is a hired penetration tester authorised by TechCorp to find weaknesses before real criminals do. Every technique here is standard OWASP and PTES methodology used by certified professionals daily. In India, authorised Red Team exercises are governed by CERT-In cybersecurity guidelines.



3.1 Attack 1 — Port Scanning with Nmap (Screenshot 2, Kali 20:37)

Definition: `nmap -sV 192.168.152.5`

Full Name: Network Mapper

Category: Open-source network discovery and security auditing tool

Syntax: `nmap [flags] <target-IP>`

Definition: Sends crafted TCP/UDP packets to each port on the target and analyses responses to determine: (1) which ports are open, (2) which service runs on each port, and (3) the exact software version. This is the foundational first step of every penetration test.

Flag: `-sV` — Service and Version detection. After finding open ports, Nmap sends additional probes to identify the exact software name and version number.

Real-World Scenario: Security Consultant Mapping TechCorp's Building Entry Points

Before a physical security auditor can test TechCorp's building, they walk the perimeter noting every door, window, and emergency exit. Each is a potential entry point. Nmap does this for the server: it knocks on all 65,535 "doors" (ports) and records which ones open. Finding `80/tcp open Apache httpd 2.4.52` is like discovering the front door is unlocked and reading the receptionist's name badge. The exact version number enables a real attacker to look up CVE-2021-41773 and other known vulnerabilities. This is why production servers suppress their version strings with `ServerTokens Prod`.

```

(cipherwolfkali)-[~]
$ nmap -sV 192.168.152.5
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 19:48 +0530
Nmap scan report for 192.168.152.5
Host is up (0.0052s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http  Apache httpd 2.4.52 ((Ubuntu))
MAC Address: 08:00:27:00:BF:21 (Oracle VirtualBox virtual NIC)

Service detection performed.
  
```

```
Nmap done: 1 IP address (1 host up) scanned in 14.02 seconds
```

Port	State	Version	Security Implication
80/tcp	OPEN	Apache httpd 2.4.52 (Ubuntu)	Exact version exposed. Attacker can search the CVE database. No HTTPS means all traffic is unencrypted.

3.2 Attack 2 — Directory Enumeration with Dirb (Screenshot 2, Kali 20:37)

Definition: `dirb http://192.168.152.5`

Full Name: Directory Buster / Web Content Scanner

Category: Web application reconnaissance tool

Syntax: `dirb <URL> [wordlist]`

Definition: Makes HTTP GET requests for every word in a built-in wordlist (4,612 common directory and file names) and reports which ones receive any response. Used to discover hidden endpoints not linked from the main page: admin panels, backup files, config pages, test directories.

Response codes:

- CODE:200 — page exists and is accessible
- CODE:403 — page exists but access is forbidden (endpoint location still revealed)

Real-World Scenario: Locksmith Testing Every Door on TechCorp's Floor

After finding the front door (port 80), the attacker tries every office and corridor door on that floor. Dirb tries 4,612 keys one by one and notes which doors exist. Finding `/server-status` at CODE:403 is significant — the door exists and is labelled, even though it is locked. The attacker now knows Apache's monitoring endpoint is enabled. In real-world attacks, Dirb-style tools have discovered exposed `/.git/` repositories containing source code, `/.env` files with database passwords, and `/admin/` panels with default credentials. None were linked from the main page.

```
(cipherwolfkali)-[~]
$ dirb http://192.168.152.5
-----
DIRB v2.22
By The Dark Raver
-----
START_TIME: Tue Mar 17 19:49:43 2026
URL_BASE: http://192.168.152.5/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

---- Scanning URL: http://192.168.152.5/ ----
+ http://192.168.152.5/index.html (CODE:200|SIZE:857)
+ http://192.168.152.5/server-status (CODE:403|SIZE:278)

END_TIME: Tue Mar 17 19:50:26 2026
```

DOWNLOADED: 4612 - FOUND: 2

3.3 Attack 3 — Traffic Flood with Curl (Screenshot 2, Kali 20:37)

Definition: `for i in {1..200}; do curl -s http://192.168.152.5 > /dev/null; done`

for loop — A Bash shell construct that repeats a command a fixed number of times. `{1..200}` generates numbers 1 to 200, running the body 200 times.

curl -s — The `-s` (silent) flag suppresses progress and error messages, making the loop run at maximum speed.

> /dev/null — Redirects the HTML response to `/dev/null` (Linux's discard device). The request is still sent and logged by the server — the output is just discarded to keep the terminal clean.

Combined effect: Sends 200 HTTP GET requests in rapid automated succession — a volume physically impossible for any human to produce manually. Creates a statistical anomaly that the Blue Team will detect instantly.

Real-World Scenario: Robot Pressing TechCorp's Doorbell 200 Times Per Second

A normal visitor rings the doorbell once. A legitimate employee visits the server 5 times a day. The curl flood is like a robot pressing the doorbell 200 times in seconds — impossible for a human. In TechCorp's SOC, this creates an unmissable spike in the access log. Real attackers use this technique for DDoS reconnaissance, WAF bypass testing, web scraping at scale, and login brute-force attacks. The pattern of 19,369 identical requests from one IP in a single session is what SIEM platforms (Splunk, IBM QRadar, Microsoft Sentinel) auto-alert on within 60 seconds.

```
(cipherwolfkali)-[~]
$ for i in {1..200}; do curl -s http://192.168.152.5 > /dev/null; done

(cipherwolfkali)-[~]
$ curl http://192.168.152.5
<!DOCTYPE html>
<html>
<head>
  <title>Red vs Blue Team Simulation</title>
</head>
<body style="font-family: Arial; text-align: center; background-color: #f4f4f4;">
  <h1> Cyber Security Lab Project</h1>
  <h2>Red Team vs Blue Team Simulation</h2>
  <p><strong>Name:</strong> Saidinesh</p>
  <p><strong>Batch:</strong> 13</p>
```

IP Address	Requests	Source	Traffic Type	
192.168.152.3 (Kali)	19,369	Attacker	Attack flood	
192.168.152.5 (Lite)	200	Server self	System traffic	
192.168.152.6 (Mint)	5	Legitimate user	Normal browsing	

4 Legitimate User Activity — Linux Mint

Linux Mint — The Normal Employee

Linux Mint plays TechCorp's HR manager who checks the company portal each morning. Five requests versus 19,369 makes the attack signature immediately obvious. After the UFW block is applied, this VM proves the rule is perfectly selective — blocking only the attacker while leaving all other users untouched.

curl from Linux Mint (Screenshot 11, 20:44):

```
user@user-VirtualBox:~$ curl http://192.168.152.5
<!DOCTYPE html>
<html>
<head>
  <title>Red vs Blue Team Simulation</title>
</head>
<body style="font-family: Arial; text-align: center; background-color: #f4f4f4;">
  <h1> Cyber Security Lab Project</h1>
  <h2>Red Team vs Blue Team Simulation</h2>
  <p><strong>Name:</strong> Saidinesh</p>
  <p><strong>Batch:</strong> 13</p>
  <hr>
  <h3> Red Team Activities</h3>
  <p>Performed Reconnaissance using Nmap</p>
  <p>Directory Enumeration using Dirb</p>
  <p>Simulated Attack using Curl Requests</p>
  <h3> Blue Team Activities</h3>
  <p>Monitored Logs using Apache Access Logs</p>
  <p>Detected Attacker IP</p>
  <p>Blocked Attacker using Firewall (UFW)</p>
  <h3> Simulation Note</h3>
  <p>This project was performed individually by acting as both attacker and defender
  .</p>
</body>
</html>
```

5 Blue Team Activities — Detection and Defense

Blue Team Role — SOC Analyst

Apache's `access.log` is TechCorp's CCTV recording. Every request, IP, and timestamp is captured automatically. The SOC analyst watches the live feed, spots the flood, identifies the attacker's IP with AWK, and applies a surgical UFW rule. All other users remain completely unaffected.



5.1 Step 1 — Real-Time Log Monitoring (Screenshots 5 and 7)

Definition: `tail -f /var/log/apache2/access.log`

tail — Prints the last lines of a file. By default, shows the last 10 lines.

-f flag — Follow. Keeps the file open and continuously prints new lines as they are appended in real time. Without `-f`, `tail` shows the last 10 lines and exits.

/var/log/apache2/access.log — Apache's access log. Every HTTP request is automatically written here with: client IP, timestamp, HTTP method, URL, status code, response size, and user-agent string.

Log format: IP - - [date] "METHOD /path HTTP/ver" status size "-" "user-agent"

Real-World Scenario: SOC Analyst Watching Live CCTV Footage at TechCorp

Without `-f`, the analyst would have to manually re-open the log file every few seconds — like rewinding a tape repeatedly. With `-f`, every new request appears on screen the instant Apache receives it — live surveillance. In TechCorp's real SOC, this feed would be piped into a SIEM platform (Splunk, Microsoft Sentinel) which auto-alerts when any single IP exceeds 100 requests per minute. In our lab, the analyst would notice within 2 seconds that **192.168.152.3** is firing **multiple requests at the exact same timestamp 19:50:56** — physically impossible for a human. The user-agent `curl/8.18.0` (a scripting tool, not a browser) confirms automated behaviour.

user	/var/www/html	tail -f /var/log/apache2/access.log
192.168.152.3	- -	[17/Mar/2026:19:50:56 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl/8.18.0"
192.168.152.3	- -	[17/Mar/2026:19:50:56 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl/8.18.0"
192.168.152.3	- -	[17/Mar/2026:19:50:56 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl/8.18.0"
192.168.152.3	- -	[17/Mar/2026:19:50:56 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl/8.18.0"
192.168.152.3	- -	[17/Mar/2026:19:50:56 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl/8.18.0"

```

192.168.152.3 - - [17/Mar/2026:19:50:56 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl
/8.18.0"
192.168.152.3 - - [17/Mar/2026:19:50:56 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl
/8.18.0"
192.168.152.3 - - [17/Mar/2026:19:50:56 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl
/8.18.0"
192.168.152.3 - - [17/Mar/2026:19:50:57 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl
/8.18.0"
192.168.152.3 - - [17/Mar/2026:19:50:57 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl
/8.18.0"
192.168.152.6 - - [17/Mar/2026:19:59:38 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl
/8.5.0"
192.168.152.6 - - [17/Mar/2026:20:00:38 +0530] "GET / HTTP/1.1" 200 807 "-" "Mozilla
/5.0 (X11; Linux x86_64; rv:146.0) Gecko/20100101 Firefox/146.0"
192.168.152.6 - - [17/Mar/2026:20:00:40 +0530] "GET /favicon.ico HTTP/1.1" 404 491 "
http://192.168.152.5/" "Mozilla/5.0 (X11; Linux x86_64; rv:146.0) Gecko/20100101
Firefox/146.0"
192.168.152.6 - - [17/Mar/2026:20:02:00 +0530] "GET / HTTP/1.1" 200 807 "-" "Mozilla
/5.0 (X11; Linux x86_64; rv:146.0) Gecko/20100101 Firefox/146.0"
192.168.152.3 - - [17/Mar/2026:20:24:50 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl
/8.18.0"
192.168.152.3 - - [17/Mar/2026:20:27:49 +0530] "GET / HTTP/1.1" 200 1109 "-" "curl
/8.18.0"
^C

```

5.2 Step 2 — Attacker Identification with AWK (Screenshots 7 and 8)

Definition: `awk '{print $1}' access.log | sort | uniq -c | sort -rn`

awk — A text-processing language. `'{print $1}'` extracts field 1 (the IP address — the first word in each log line) from every line in the file.

`|` (pipe) — Passes the output of one command as input to the next.

sort — Alphabetically sorts the list of IPs so identical ones are grouped.

uniq -c — Counts consecutive identical lines. `-c` prepends the count.

sort -rn — Sorts numerically (**n**) in reverse (**r**) order. Highest count appears at the top — that is always the attacker.

Combined effect: Counts requests per IP address and ranks them highest first in a single command pipeline.

Real-World Scenario: Counting Visitor Badges at TechCorp's Reception After an Incident

After a security incident, the reception desk has a visitor log book (the access log). The security team uses **awk** to extract just the visitor names (the IP column), **sort** to group all visits by the same person, **uniq -c** to count each person's visits, and **sort -rn** to rank from most to least. The result is immediate: 19,369 visits from one IP vs 5 from another. This same AWK pipeline is used daily by SOC analysts at HDFC Bank, Infosys, and TCS to triage security incidents from millions of log lines in seconds — no expensive SIEM tool required.

```

user /var/www/html SIGINT
user /var/www/html

```

```
awk '{print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -rn
19369 192.168.152.3
200 192.168.152.5
5 192.168.152.6
```

IP Address	Count	Classification	Action	
192.168.152.3	19,369	Attacker	BLOCK immediately	
192.168.152.5	200	Server self	Monitor only	
192.168.152.6	5	Legitimate user	Allow — do not touch	

5.3 Step 3 — Install and Verify UFW (Screenshot 8)

Definition: `sudo apt update && sudo apt install ufw -y`

UFW — Uncomplicated Firewall. A user-friendly frontend for Linux's `iptables` kernel packet filter. Provides simple commands to allow or deny network traffic without writing complex low-level rules directly.

sudo apt update — Refreshes the local package index so apt knows about the latest available versions.

Output confirms: `ufw` is already the newest version (0.36.1-4ubuntu0.1) — UFW was pre-installed with Ubuntu and needed no download.

```
user /var/www/html sudo apt update
user /var/www/html sudo apt install ufw -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ufw is already the newest version (0.36.1-4ubuntu0.1).
0 upgraded, 0 newly installed, 0 to remove and 558 not upgraded.

user /var/www/html sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
80 ALLOW IN Anywhere
80 (v6) ALLOW IN Anywhere (v6)
```

5.4 Step 4 — Block the Attacker with UFW (Screenshots 9 and 10)

Definition: `sudo ufw deny from 192.168.152.3`

ufw deny from <IP> — Creates a firewall rule that silently drops ALL packets arriving from the specified IP address. The attacker receives no error message; their connections simply time out (133 seconds in this lab).

DENY vs REJECT: DENY silently drops packets — the attacker does not know why it failed. REJECT sends back an error message. DENY is preferred in production because it leaks less information to the attacker.

ufw reload — Applies all pending rule changes immediately.

ufw status numbered — Lists all active rules with index numbers. These numbers are required to delete individual rules with `ufw delete <number>`.

Real-World Scenario: TechCorp Banning One Specific Visitor from the Building

After identifying the attacker's face on CCTV, TechCorp adds **192.168.152.3** to the banned visitor list. The next time this person approaches, the automatic doors simply do not open — no explanation, just silence. They wait 133 seconds and give up. Meanwhile every other employee's badge works perfectly. The HR manager at **192.168.152.6** walks in without any delay. In production, this same pattern is how Cloudflare and AWS Shield block millions of malicious IPs daily — precise, selective, and instant.

```

user /var/www/html sudo ufw deny from 192.168.152.3
Rule added

user /var/www/html sudo ufw status verbose
Status: active
To Action From
--
80 ALLOW IN Anywhere
Anywhere DENY IN 192.168.152.3
80 (v6) ALLOW IN Anywhere (v6)

user /var/www/html sudo ufw deny from 192.168.152.3
user /var/www/html sudo ufw reload
Skipping adding existing rule
Firewall reloaded

user /var/www/html sudo ufw status numbered
Status: active

To Action From
--
[ 1] 80 ALLOW IN Anywhere
[ 2] Anywhere DENY IN 192.168.152.3
[ 3] 80 (v6) ALLOW IN Anywhere (v6)

```

Rule cleanup and reordering so DENY is evaluated first:

```

user /var/www/html sudo ufw delete 1
Deleting:
allow 80

```

```

Proceed with operation (y|n)? y
Rule deleted

user /var/www/html sudo ufw delete 2
Deleting:
allow 80
Proceed with operation (y|n)? y
Rule deleted (v6)

user /var/www/html sudo ufw delete 3
ERROR: Could not find rule '3'

user /var/www/html sudo ufw deny from 192.168.152.3
Skipping adding existing rule

user /var/www/html sudo ufw status numbered
Status: active
      To      Action      From
[ 1] Anywhere  DENY IN    192.168.152.3

user /var/www/html sudo ufw allow 80
Rule added
Rule added (v6)

user /var/www/html sudo ufw reload
Firewall reloaded

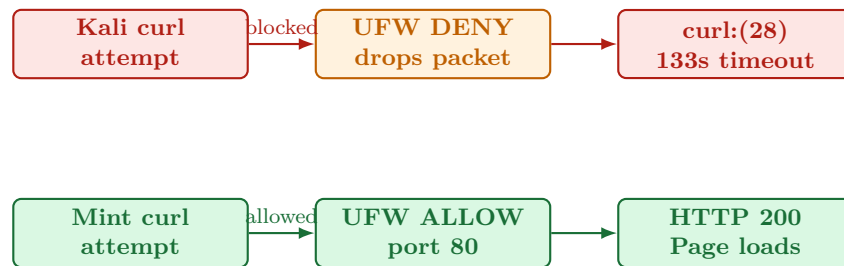
user /var/www/html sudo ufw status numbered
Status: active
      To      Action      From
--      -
[ 1] Anywhere  DENY IN    192.168.152.3
[ 2] 80        ALLOW IN    Anywhere
[ 3] 80 (v6)    ALLOW IN    Anywhere (v6)

```

Why Rule Order Matters

Rule [1] DENY IN from **192.168.152.3** is evaluated **first**. All packets from Kali are dropped silently before they ever reach Rule [2]. Rules [2] and [3] keep port 80 open for all other IPs including Linux Mint. This ordering is identical to how Cloudflare and AWS WAF place deny rules above allow rules to block attackers before whitelist checks are applied.

6 Results and Verification

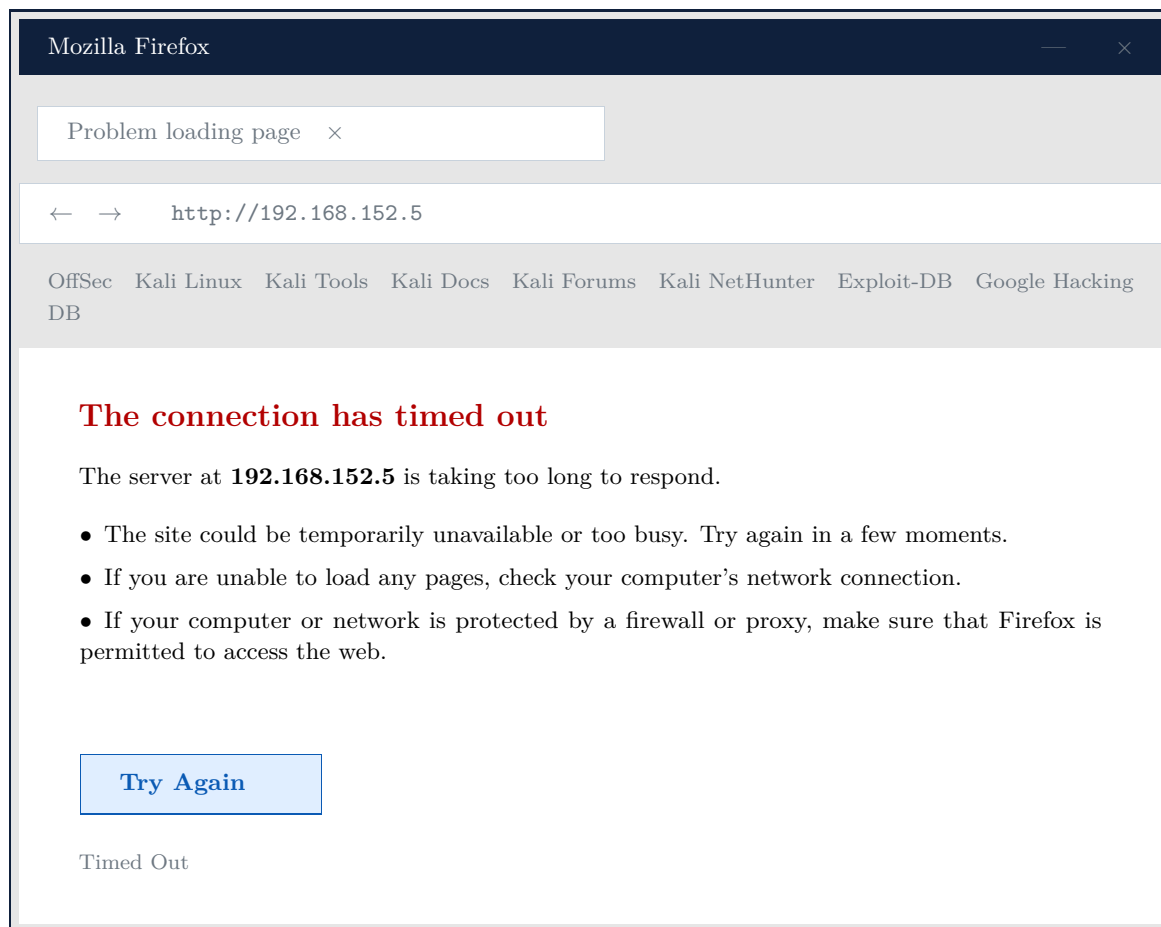


6.1 Kali Blocked — curl Test (Screenshot 3)

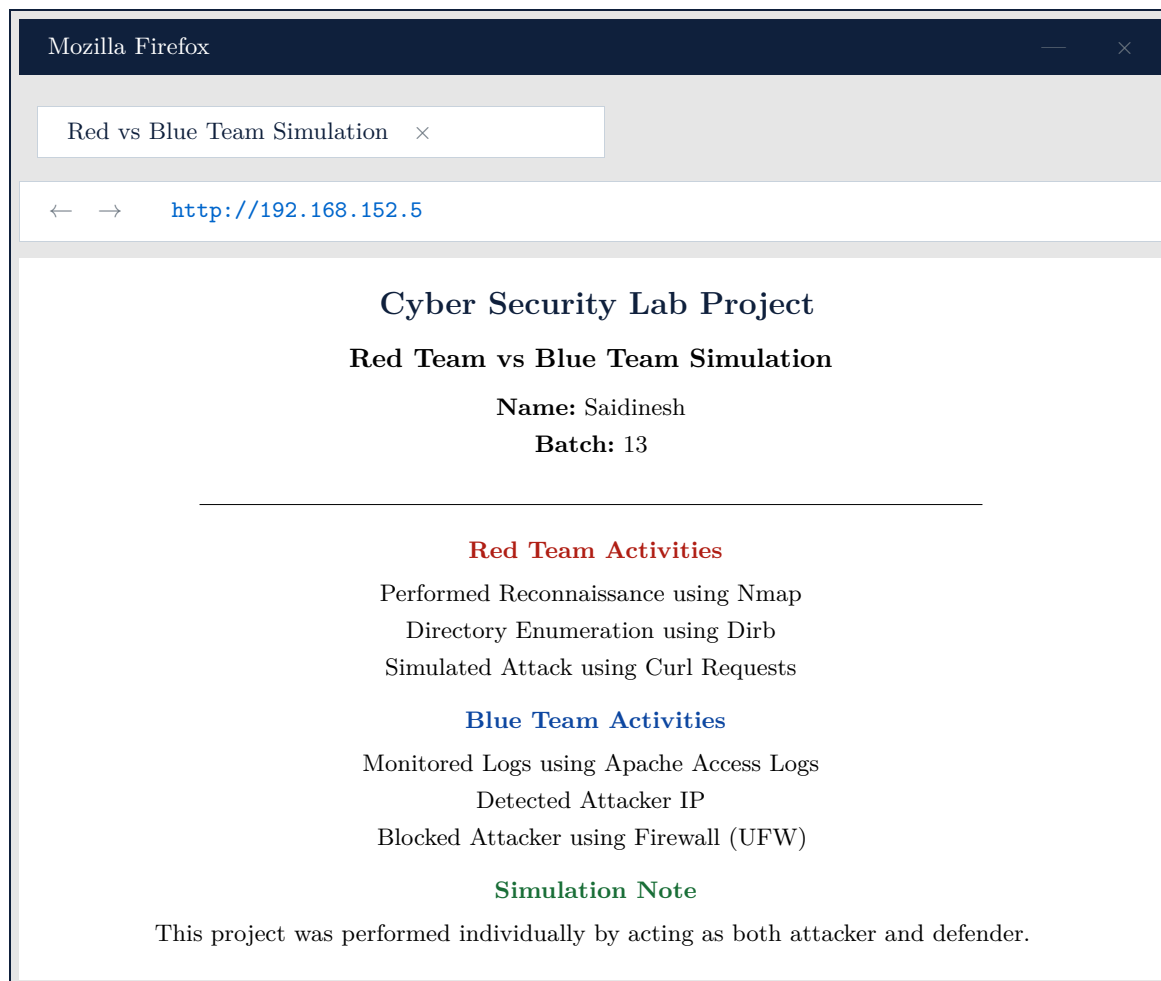
```
(cipherwolfkali)-[~]  
$ curl http://192.168.152.5  
curl: (28) Failed to connect to 192.168.152.5 port 80 after 133480 ms: Could not  
connect to server
```

Error 28 = connection timeout. UFW silently drops every packet from **192.168.152.3**.

6.2 Kali Blocked — Firefox Browser (Screenshot 4)



6.3 Linux Mint Unaffected — Firefox Success (Screenshot 12)



6.4 Final Test Results

#	Test Case	Result
1	Kali curl blocked: <code>curl: (28) Failed to connect after 133480 ms</code>	PASS
2	Kali Firefox: "The connection has timed out"	PASS
3	Linux Mint curl returns full HTML page (HTTP 200)	PASS
4	Linux Mint Firefox loads the full styled page	PASS
5	AWK confirms 19,369 requests from <code>192.168.152.3</code>	PASS
6	<code>ufw status numbered</code> : DENY IN <code>192.168.152.3</code> is rule [1]	PASS

All Tests Passed — Mission Complete

- Attacker detected via log analysis: 19,369 requests from one IP confirmed
- Attacker blocked at kernel level: curl timeout (133 s) and Firefox timeout confirmed
- Legitimate user (Linux Mint) completely unaffected throughout the exercise
- UFW DENY rule surgically distinguishes attacker from all legitimate traffic
- Full SOC incident response lifecycle replicated end-to-end on real virtual machines

7 Conclusion and Key Learnings

7.1 Technical Skills Demonstrated

Skill	What Was Done	Industry Role
Network Reconnaissance	Nmap 7.98 revealed Apache 2.4.52 on port 80	Pen Tester
Web Enumeration	Dirb discovered <code>/index.html</code> and <code>/server-status</code>	Web App Tester
Traffic Analysis	19,369 request flood created unmistakable anomaly	Threat Analyst
Log Analysis	AWK pipeline identified attacker from thousands of log lines	SOC Analyst
Firewall Management	UFW DENY blocked one IP, all others unaffected	NetSec Engineer
Incident Response	detect → analyse → contain → verify	DFIR Analyst
Lab Engineering	3-VM isolated VirtualBox environment	SecOps / DevSecOps

7.2 Key Learnings

1. **Attackers always leave traces.** Every request is logged with IP, timestamp, and user-agent. There is no invisible attack at the network layer.
2. **Volume anomalies are the simplest signatures.** 19,369 vs 5 requests — one AWK command finds the attacker from thousands of log lines instantly.
3. **Exact versions enable targeted attacks.** Nmap exposed Apache `httpd 2.4.52`, enabling CVE lookups. Version banners must be hidden in production.
4. **A 403 still leaks endpoint existence.** `/server-status` returned 403 but its location is now known to the attacker — a common misconfiguration.
5. **Firewall rules can be surgical.** One UFW DENY blocked a single IP with zero impact on any other traffic.
6. **Dual-role simulation builds 360-degree understanding.** Performing both Red and Blue roles reveals how attack decisions directly produce log signatures.

7.3 Real-World Application

Industry	How These Techniques Are Used Daily
Banking (HDFC, SBI)	AWK-style log analysis on transaction servers detects account takeover and fraud
E-Commerce (Amazon, Flipkart)	Request-count analysis blocks scrapers stealing pricing data at scale
Government / Defence	Red Team exercises identical to this lab are mandated under CERT-In guidelines
Cloud (AWS, Azure, GCP)	UFW-style rules power VPC security groups protecting millions of servers
Cybersecurity Firms	Scan → enumerate → attack → detect → block → verify is the full penetration-test engagement lifecycle

Final Statement

This lab successfully replicated a complete cybersecurity incident lifecycle:

reconnaissance → enumeration → exploitation → detection → containment → verification

By performing both attacker and defender roles in the same controlled environment, this exercise built a 360-degree understanding of offensive and defensive security operations that is rare and highly valued in the industry.

Skills demonstrated: Network Reconnaissance | Web Enumeration | Traffic Flood Analysis | Log Analysis | Intrusion Detection | Firewall Configuration | Incident Response